

第一章：项目引言与总体概述

1.1 项目开发背景与核心定位

本项目是一款基于 Unity 引擎与 Python 后端技术栈开发的 3D 网络联动网络闯关游戏。项目突破了传统单机期末作业的局限性，在实现角色物理控制器、多关卡管理闭环的基础上，引入了全异步的 HTTP 后端网络服务体系。系统底层旨在解决多账号数据物理隔离、高并发测试环境下的网络状态管理，以及跨场景内存数据持久化等软件工程痛点。

1.2 系统核心技术架构

本系统整体采用 C/S (Client/Server, 客户端/服务端) 网络架构 进行设计，主要由以下三大核心层级构建而成：

- **表现与物理表现层 (Unity 客户端)**：利用 CharacterController 构建具备高精度运动学的物理控制器；采用单例模式的 GameStateManager 实现逻辑时间轴切换与多关卡无缝跳转。
- **网络中转与缓存解耦层 (NetworkManager 全局单例)**：基于 Unity 提供的 UnityWebRequest 模块，封装了全异步的 HTTP 状态机呼叫逻辑，并利用全局内存锁 (DontDestroyOnLoad) 确保网络回执数据不随场景切换而崩塌。
- **业务逻辑与持久化存储层 (Python 后端)**：采用轻量级高性能框架 Flask 搭建本地高响应测试服务器。放弃了传统的重型数据库（如 MySQL），创新性地设计了“用户名-JSON 文件”的轻量级无序映射存储模型 (NoSQL 思想)，实现了零污染的多账号物理隔离存档。

第二章：客户端单机核心系统设计

2.1 基于运动学的角色控制器 (PlayerController)

游戏主角（兔子）的移动与物理交互是单机玩法的核心。本项目拒绝了容易产生穿模和碰撞抖动的 Rigidbody 刚体物理，转而采用具备高度可控性的 CharacterController（角色控制器）作为物理底座。

- **水平运动学控制**：系统在 Update 每帧动态捕捉玩家的键盘输入 (Horizontal / Vertical 轴向数值)，通过向量乘法转化为 3D 世界坐标系下的移动步长，保证了操作的即时响应与转向的平滑度。
- **重力与精准跳跃算法**：为了克服单机重力模拟不稳定的隐形 Bug，脚本引入了基于物理学自由落体反向运动的数学公式。当玩家触发跳跃时，通过公式 $v = \sqrt{2gh}$ 逆向求解出角色所需的垂直向上初速度 v （其中 g

为自定义重力加速度， h 为目标跳跃高度），随后在每帧递减垂直速度以模拟真实的抛物线跳跃体验。

2.2 状态判定与时间轴死锁控制（GameStateManager）

关卡流转与结算控制采用单例模式（Singleton Pattern）构建了 **GameStateManager**。该模块核心解决了 Unity 切换场景时常见的数据冲突与时间轴死锁问题。

- **生命周期与单例唯一性**：通过 **Awake** 阶段的实例检查，确保场景中永远有且仅有一个游戏状态主管，任何重复生成的总管物体都将被强行销毁，以此规避全局调用错乱。
- **时间轴暂停控制（Time.timeScale）**：当玩家触发胜利（抵达终点）或失败（血量归零/坠落）时，系统通过调整 **Time.timeScale = 0f** 彻底冻结全场的物理演算与动作状态机，呼出结算面板。在玩家点击“重新开始”或“加载下一关”时，系统在执行 **SceneManager.LoadScene** 前会强制将 **Time.timeScale** 恢复为 **1f**，彻底避免了因时间轴未重置导致的新关卡卡死现象。

第三章：网络与服务端（Backend）系统设计

3.1 基于 HTTP POST 协议的异步身份验证机制

为了实现轻量级、高响应的客户端与服务端联动，项目放弃了复杂的 TCP 长连接，采用了基于 **HTTP** 协议的短连接全异步通信架构。

- **表单构建与传输**：客户端通过 Unity 的 **WWWForm** 组件将玩家输入的账号与密码进行字典包装，利用 **UnityWebRequest.Post** 异步发送至后端的 **/login** 路由端口。
- **网络防假死优化（Timeout）**：为了防止因为网络波动或防火墙静默拦截导致的协程无限期挂起（假死）现象，客户端网络层强制引入了 **webRequest.timeout = 3** 的断路器机制，确保在 3 秒内未收到握手回执时能够安全熔断并向玩家弹出提示。

3.2 基于多账户的客户端独立表现隔离

“多账号登录进同一款游戏却展现截然不同的初始状态”，是本项目的一大技术亮点（满足考核指标）。

- **业务流设计**：服务端收到登录请求后，校验内存中的虚拟数据库（**USER_DB**）。若验证通过，则根据不同的用户名下发定制化的

JSON 数据包（例如账号 `alice` 对应高敏捷号，下发速度 12.0、药水 15；账号 `bob` 对应常规体验号，下发速度 5.0、药水 2）。

- **动态解耦解析：**客户端通过 `JsonUtility.FromJson` 将下发的 JSON 字符串反序列化为 C# 实体对象。随后通过“网络雷达搜索”捕获当前场景中的兔子对象，强行用服务器数据覆盖本地单机默认值，从底层实现了“一套单机逻辑，多账号千人千面”的隔离表现。

3.3 无数据库环境下的多文件 JSON 存档持久化

针对大作业轻量化交付、不便配置重型数据库（如 MySQL/Oracle）的限制，本项目精巧地实现了“无序映射文件级持久化存档方案”。

- **文件存储闭环：**在服务端开辟独立的 `player_datas` 本地物理文件夹。每当新账号首次登录时，系统会自动在磁盘上为其创建名为 `user_用户名.json` 的专属数据文件。
- **实时覆盖写入（数据存储）：**当玩家在游戏内捡起锦囊（数量改变）或通过 GM 修改器面板更新配置时，客户端会瞬间调用 `SaveCurrentDataToServer` 接口将最新快照上传。服务端接收后，利用 Python 的文件流写入机制对该账号的 `.json` 存档执行强行擦除并重写（**Overwrite**）。此设计完美做到了“无需庞大数据库支撑，数据 100% 永久留存”的工业级持久化效果。

第四章：系统测试与项目结论

4.1 测试用例与运行结果表现

为验证系统的稳定性和数据准确性，项目组对核心网络功能进行了全方位的黑盒测试：

1. **老用户登录测试：**输入账号 `alice` 密码 `123456`，Unity 成功跳转至主菜单；进入关卡后，兔子移动速度为云端存储的 12.0，背包药水为 15，控制台日志链路完整。
2. **数据持久化测试：**在登录 `alice` 后，通过 GM 面板将兔子速度调整为 18。此时观察服务端 `player_datas` 文件夹，发现 `user_alice.json` 内部的 `playerSpeed` 字段已实时变更为 18。切换场景并重新进入后，兔子速度依然保持为修改后的 18，证明云端数据覆盖与持久化读取完全成功。
3. **隔离性与安全性测试：**输入账号 `bob` 登录，其速度与背包数量完全不受 `alice` 的数据污染，严格保持其专属的独立存档，证明多账号隔离无误。输入错误密码时，系统安全熔断并返回 401 状态码，客户端提示登录失败。

4.2 项目总结

本项目紧密围绕期末大作业的各项考核指标，成功将一个传统的单机 3D 闯关游戏升级为了具备 C/S 架构的网络联动游戏。通过对 DontDestroyOnLoad 全局单例、网络异步协程、雷达解耦搜索以及 Flask 多文件读写等技术的综合运用，切实解决了跨场景数据丢失、数据物理隔离与文件级持久化等核心问题。系统运行流畅，架构逻辑严密，达到了预期的软件设计与交付标准。

第五章：AI Agent 协同开发流程与实验环境配置

5.1 实验开发与运行环境

为保证系统在客户端与服务端的双向联通及稳定运行，本项目搭建了如下标准的软件工程实验环境：

- 开发操作系统：Windows 11
- 前端开发引擎：Unity 2022.3 LTS
 - 核心组件：CharacterController（角色控制器）、UnityWebRequest（网络通信模块）、TextMeshPro（UI 文字渲染）。
- 后端开发环境：PyCharm 2024
 - 核心依赖包：Flask 3.0+（轻量级 Web 服务器框架）、os & json（标准文件流处理库）。
- 网络测试拓扑：本地回环网络（Loopback），标准监听地址为 `http://localhost:5000`，全异步 HTTP 短连接。

5.2 AI Agent (Cursor) 协同设计与敏捷开发流程

本项目引入了前沿的 **AI Agent (Cursor)** 协同编程范式，将传统的“人工查阅 API → 手写代码 → 逐行 Debug”的死板流程，升级为了“意图输入 → Agent 架构生成 → 动态感知重构”的敏捷协同开发流。

1. **架构设计与代码生成（Generation Stage）**：在单机向网络化转型的初期，利用 Cursor 对项目全局上下文的感知能力（Context-Awareness），通过自然语言交互直接完成了 NetworkManager.cs 底层 HTTP 状态机的全异步架设。Agent 自动引入了 C# 协程（Coroutine）与 WWWForm 字典包装技术。
2. **代码重构与解耦（Refactoring Stage）**：针对 Unity 经典的“跨场景网络请求断裂与内存重置”大坑，在 Agent 的协助下对底层架构进行了“全局内存锁（DontDestroyOnLoad 单例）+ 关卡拉取流（Pull Architecture）”的解耦重构。Agent 创新性地引入了“网络雷达搜索”机制，使物品捡拾脚本（InventoryManager.cs）能够越过单例依赖，直接通过 Tag 动态捕捉场景中的兔子对象，完成持久化上传。

3. 动态调试与错误熔断（Debugging & Troubleshooting）：

在实验运行阶段，由于操作系统防火墙及本地 IPv6 优先权问题，Unity 端曾出现长达数秒的网络静默挂起（假死）。通过与 Cursor Agent 的联合排查，动态调整了代码策略：

- 在客户端全线部署了 `webRequest.timeout = 3` 断路器熔断机制。
- 将硬编码的 IP 地址对齐为兼容性更强的 `localhost` 本地回环。
- 在 `NetworkManager` 销毁期加入 `StopAllCoroutines()` 强制清空 GC 句柄（Handle），彻底攻克了 `Release of invalid GC handle` 的内存泄漏隐患。

5.3 核心技术 Demo 演进对比

为了向评审老师直观展示 AI Agent 在本项目中的生产力提升，系统架构经历了三个核心 Demo 阶段的演进：

- **Demo 1.0（纯单机基础版）**：实现了兔子的水平运动、物理跳跃公式 $v = \sqrt{2gh}$ 以及最基础的关卡切换。此时数据完全依赖内存，关卡重启或切换时数值会被瞬间重置（无持久化能力）。
- **Demo 2.0（网络隔离过渡版）**：引入 Python Flask 后端。实现了主菜单登录鉴定，服务器可根据 `alice` 和 `bob` 的身份下发不同的 JSON 配置，客户端实现了解耦解析与速度/药水的表现隔离。但此时数据仅能“单向下载”，玩家在游戏内收集的锦囊无法回传服务器。
- **Demo 3.0（双向持久化完整版）**：在 Cursor Agent 协同下完成了**数据上传与覆盖擦写闭环**。玩家无论是触发 GM 修改器还是捡起物品，数据都能瞬间上传并永久改写服务端的 `user_用户名.json` 文件。至此，无数据库环境下的“千人千面、永久存档”完美收官。